

Cardiix (Vivitsu Medical AI / CardiaX)

AI-Powered Early Cardiovascular Risk Screening & Smart Lipid Monitoring System

Contents

Executive Overview	1
System Architecture	3
Deep Dive into Codebase & Core Components	4
1. Remote Photoplethysmography (rPPG) & Vital Detection Engine	4
2. Corneal Arcus Eye Analysis Engine	5
3. Smart Medical Report Analyzer & OCR Engine	5
4. Context-Aware AI Health Assistant (“Vivitsu”)	5
5. Biometric IoT Integration	6
Database Schemas & Data Models	7
MongoDB Models	7
Supabase Realtime Table — health_readings	7
Tech Stack Overview	8
UI Application Views	9
Medical Disclaimer & Compliance	10
Key Highlights & Innovations	10

Executive Overview

Cardiix (also referred to in the codebase as Vivitsu Medical AI or CardiaX) is a multi-modal, AI-driven healthcare platform designed for early cardiovascular disease (CVD) screening, non-invasive vital signs detection, smart lipid report analysis, and continuous IoT tele-monitoring.

Cardiovascular diseases are the leading cause of mortality globally, often remaining asymptomatic until a severe cardiac event occurs. Traditional diagnostic checkups require invasive blood draws, expensive laboratory setups, and manual report evaluations. Cardiix addresses these barriers by combining:

- **Remote Photoplethysmography (rPPG)** via webcam video to detect Heart Rate (BPM), Heart Rate Variability (HRV), estimated Blood Pressure (BP), and Stress levels without physical contact.
- **Corneal Arcus Computer Vision Detection** via high-resolution eye imagery to identify Arcus Senilis (peripheral corneal lipid rings) as a non-invasive visual screening marker for hypercholesterolemia.
- **Automated Medical & Lipid Report OCR Parsing** with Large Language Model (LLM) clinical reasoning to extract total cholesterol, LDL, HDL, and triglycerides.

- **IoT Telemetry Streaming** via ESP32 microcontrollers and MAX30102 pulse oximeters to push real-time BPM and SpO₂ data into a cloud database.
- **Context-Aware AI Medical Assistant** powered by Groq's high-speed inference engine (openai/gpt-oss-120b) and Google Gemini 2.5 Pro.

System Architecture

Frontend Layer — React 19 + Vite + TypeScript Dashboard / VitalScan / ReportAnalyzer / ChatAssistant UI, with a jsPDF + html2canvas exporter for report downloads.

IoT & Hardware Layer ESP32 microcontroller paired with a MAX30102 pulse oximeter streams data via HTTP POST /data or directly into Supabase, feeding the IoT database.

Python Microservices — FastAPI + OpenCV + SciPy

- *BPM Service* (Port 8000): POS algorithm + FFT + Butterworth filter
- *Eye Service* (Port 5004): Inpainting + Hough Circles + 12-segment spatial CV

Backend Engine — Node.js Express (Port 5001) Express server coordinating a Tesseract OCR engine and a MongoDB database.

Cloud & AI Layer Groq API (gpt-oss-120b), Google Gemini 2.5 Pro Vision, and Supabase PostgreSQL (Realtime).

Data flow summary:

1. UI sends a recorded video (WebM) to the BPM microservice for rPPG analysis.
2. UI sends an eye image to the Eye microservice for corneal arcus detection.
3. UI sends a lab report image to the Node.js server, which performs OCR and forwards it to Groq for LLM clinical parsing (falling back to Gemini vision analysis if needed).
4. The IoT database syncs to Supabase, which pushes realtime updates to the UI over WebSockets.

Deep Dive into Codebase & Core Components

1. Remote Photoplethysmography (rPPG) & Vital Detection Engine

Source files: `services/backend_bpm_service.py`, `services/live_bpm_detector.py` **Framework & port:** FastAPI on `http://localhost:8000`

Methodology & Mathematical Foundation

Facial detection & forehead ROI extraction

- Uses OpenCV's Haar Cascade classifier (`haarcascade_frontalface_default.xml`).
- Isolates the forehead region ($Y_{\text{start}} = y + 0.2h$, $Y_{\text{height}} = 0.15h$, $X_{\text{start}} = x + 0.3w$, $X_{\text{width}} = 0.4w$), which has high capillary density and minimal facial muscle distortion.

Plane-Orthogonal-to-Skin (POS) algorithm (Wang et al., 2017)

Converts the time-series RGB signals (R , G , B) from the forehead ROI into a normalized signal that eliminates motion artifacts and intensity variations:

$$\begin{aligned}\bar{C}(t) &= \frac{C(t)}{\mu_C} \\ X(t) &= R_n(t) - G_n(t) \\ Y(t) &= R_n(t) + G_n(t) - 2B_n(t) \\ \alpha &= \frac{\sigma(X)}{\sigma(Y)} \\ P(t) &= X(t) - \alpha Y(t)\end{aligned}$$

Filtering & spectral frequency analysis

- Applies a 4th-order Butterworth bandpass filter (0.7 Hz - 3.5 Hz, corresponding to 42-210 BPM).
- Applies linear detrending with `scipy.signal.detrend`.
- Executes a Fast Fourier Transform (FFT) to find the peak frequency f_{peak} : $\text{BPM} = f_{\text{peak}} \times 60$.
- Calculates Signal-to-Noise Ratio (SNR) for signal quality validation ("Excellent", "Good", "Fair", "Poor").

Vitals derivative estimation

- Blood Pressure (BP):

$$\text{Systolic} = \text{clamp}(105 + (BPM - 65) \times 0.8, 90, 160)$$

$$\text{Diastolic} = \text{clamp}(70 + (BPM - 65) \times 0.5, 60, 100)$$

- Heart Rate Variability (HRV):

$$\text{HRV} = \text{clamp}(75 - (BPM - 65) \times 0.4, 25, 95) \text{ ms}$$

- **Stress Index:** synthesizes elevated HR and suppressed HRV into a 0-100 scale.

2. Corneal Arcus Eye Analysis Engine

Source file: `services/eye_service.py` **Framework & port:** FastAPI on `http://localhost:5004`

Corneal Arcus (Arcus Senilis) is a greyish-white lipid deposit around the corneal margin, serving as an overt biomarker for systemic hypercholesterolemia.

Pipeline:

1. Eye image upload
2. Specular reflection removal (inpainting bright spots above a 220 threshold)
3. Iris boundary detection (Hough Circle Transform — `cv2.HoughCircles`)
4. Spatial dual-ring masking:
 - Peripheral ring mask: $0.88r - 0.98r$
 - Inner iris mask: $< 0.70r$
 - Sclera reference point: $x + 1.3r$
5. Multi-factor signal analysis:
 - Contrast ratio = $\text{median}(\text{ring}) / \text{median}(\text{iris})$
 - Canny edge density (< 0.32 check to reject harsh glare)
 - 12-segment polar spatial uniformity ($CV < 0.35$, bright segments ≥ 8)
6. Risk classification (Strong / Moderate / Mild → Elevated / Low Risk)

Notes:

- Inpainting uses Telea inpainting (`cv2.inpaint`) to erase camera-flash reflections before color intensity evaluation.
- Debug artifact: automatically outputs a labeled debug image (`debug_arcus_detection.jpg`) illustrating iris bounds (cyan), peripheral ring (green/red), and sclera sample points (magenta).

3. Smart Medical Report Analyzer & OCR Engine

Source files: `backend/index.js`, `backend/server.js`, `components/ReportAnalyzer.tsx` **Framework & port:** Node.js Express on `http://localhost:5001`

Workflow:

1. **Document intake** — accepts base64-encoded images (JPG, PNG, WEBP) of lab results.
2. **OCR extraction** — uses `node-tesseract-ocr` (`lang: eng, oem: 1, psm: 3`) to convert physical blood-test printouts into machine-readable text.
3. **Clinical reasoning engine** — calls the Groq API (`openai/gpt-oss-120b`) with a system prompt requesting:
 - Executive summary
 - Key clinical findings
 - Numerical values & reference ranges (total cholesterol, LDL, HDL, triglycerides, fasting glucose)
 - Flagged abnormalities & risk warnings
 - Lifestyle & dietary recommendations
4. **Fallback chain:** Primary Groq API → Google Gemini 2.5 Pro → Python subprocess (`backend/medical_analyzer.py`) → local regex parser.
5. **PDF generation** — clients render and export their summary using `html2canvas` and `jsPDF`.

4. Context-Aware AI Health Assistant (“Vivitsu”)

Source files: `components/ChatAssistant.tsx`, `services/groqService.ts`

Features:

- **Dynamic context injection** — whenever the user sends a query, the service fetches all previously uploaded medical reports from MongoDB and prepends them as system context to the LLM.

- **Diet plan generation** — formulates custom cardiovascular diet strategies (e.g., Mediterranean diet focusing on soluble fibers, omega-3 fatty acids, low sodium).
- **Health trend analysis** — evaluates progressive changes across historical lipid panels over time.

5. Biometric IoT Integration

- **Hardware:** MAX30102 / MAX30100 PPG sensor paired with an ESP32 microcontroller.
- **Telemetry data stream:** measures real-time Heart Rate (BPM) and Blood Oxygen Saturation (SpO₂ %).
- **Endpoints & realtime sync:** pushes metrics via HTTP POST /data to the Node.js backend or directly inserts into the Supabase PostgreSQL table `health_readings`. The React dashboard listens to Supabase WebSocket subscriptions (`postgres_changes` on `health_readings`) to instantly stream real-time sensor graphs.

Database Schemas & Data Models

MongoDB Models

Scan Schema (backend/models/Scan.js)

```
{
  heartRate: { type: Number, required: true },
  hrv: { type: Number, required: true },
  bloodPressure: {
    systolic: { type: Number, required: true },
    diastolic: { type: Number, required: true }
  },
  stressIndex: { type: Number, default: 0 },
  aiInterpretation: { type: String, default: 'Scan completed successfully.' },
  timestamp: { type: Date, default: Date.now },
  userId: { type: String, default: 'default-user' }
}
```

MedicalReport Schema (backend/MedicalReport.js)

```
{
  uploadedImage: String,
  extractedText: String,
  aiAnalysis: String,
  uploadedAt: { type: Date, default: Date.now }
}
```

HealthData Schema (ESP32 IoT Schema)

```
{
  bpm: Number,
  spo2: Number,
  timestamp: { type: Date, default: Date.now }
}
```

Supabase Realtime Table — health_readings

Column	Type
id	primary key
bpm	Number
spo2	Number
created_at	Timestamp

Tech Stack Overview

Area	Technologies & Libraries
Frontend UI	React 19, TypeScript 5.8, Vite 6, Tailwind CSS, Lucide React Icons
Data Viz & Reports	Recharts (v3.6), jsPDF (v4.0), html2canvas (v1.4)
OCR & Image Processing	Tesseract.js (v7.0), node-tesseract-ocr
Node Backend	Express.js 5.2, Mongoose 9.1, CORS, Dotenv, Body-Parser
Python Microservices	FastAPI, Uvicorn, OpenCV (cv2), SciPy (FFT, signal processing), NumPy, Pillow
AI / LLM Integration	Groq SDK (v0.37, gpt-oss-120b), Google Gemini 2.5 Pro API (@google/genai)
Databases	MongoDB (vivitsu_health), Supabase PostgreSQL (Realtime WebSockets)
IoT / Hardware	ESP32 Microcontroller, MAX30102 / MAX30100 Pulse Oximeter

UI Application Views

Dashboard Console (`components/Dashboard.tsx`)

- Live metrics summary: Heart Rate (BPM), estimated blood pressure (mmHg), oxygen saturation (SpO₂ %).
- Dual-area interactive timeline graph powered by Recharts showing vital signal trends.
- Integrated corneal arcus eye analysis uploader and results display.
- Medical report history modal with clinical summary viewer (`ReportAnalysisModal`).
- Realtime connection diagnostics for Node.js, FastAPI, and Supabase.

Vital Scan (`components/VitalScan.tsx`)

- Interactive 30-second webcam video capture interface with live mirror feed and countdown overlay.
- Sends recorded WebM video blob to the FastAPI rPPG backend.
- Displays calculated Heart Rate, Blood Pressure, HRV, Signal Quality, and AI Verdict.
- Stores scan results to MongoDB and syncs with browser `localStorage`.

Report Analyzer (`components/ReportAnalyzer.tsx`)

- Drag-and-drop document upload interface.
- Displays real-time progress during OCR extraction and LLM clinical evaluation.
- Structured output view with a downloadable PDF report generator.

Chat Assistant (`components/ChatAssistant.tsx`)

- Full-featured conversational interface with Vivitsu Medical AI.
- Quick action triggers: “Get Diet Plan”, “Health Insights”, and “Symptom Analysis”.

Medical Disclaimer & Compliance

Cardiix is built strictly as an early risk screening, educational, and preventive health monitoring tool. **It is not a certified diagnostic medical device.** The user interface contains mandatory medical disclaimers (App.tsx disclaimer modal) reminding users to consult board-certified physicians for clinical diagnoses and medical interventions.

Key Highlights & Innovations

- **Multi-modal, non-invasive diagnostics:** combines video rPPG, computer vision eye analysis, IoT telemetry, and document OCR into a single platform.
- **Zero heavy neural nets for rPPG:** uses physics-based skin-reflection algorithms (POS) and FFT frequency decomposition, allowing high performance without demanding GPU hardware.
- **Fallback resiliency:** automatic degradation paths (Groq → Gemini → Python → local regex parser) guarantee uptime even during API outages.